Evaluating Agricultural Loans and Expenditure Allocation in Nigeria Using Double Machine Learning

William Hankins[1], Alexis Villacis[2], Ani Katchova[3], Ivette Contreras[4]

[1]The Ohio State University. Email: hankins.181@osu.edu

[2]The Ohio State University. Email: villacis.9@osu.edu

[3]The Ohio State University. Email: katchova.1@osu.edu

[4] The World Bank. Email: icontreras@worldbank.org

Contents

Reproducibility Package README

The code within this replication package executes the analysis detailed in the paper "Agricultural Loans and Expenditure Allocation in Nigeria: Evidence from Double Machine Learning" using Stata, R, and Python. This package includes all code required to go from the raw survey microdata to the replication of all tables and figures in the analysis. The Stata master file (`00_master.do`) manages sequential execution of all data construction code. Python estimation is run in a second stage, these programs are computationally taxing and can take between minutes to over a day to run depending on specifications and hardware. Our primary objective is to ensure accessibility and credibility in replicating our research methodology and results.

Data Availability and Provenance Statements

The analysis uses two rounds of Nigeria's General Household Survey – Panel (GHS-Panel), a nationally representative panel survey conducted by the National Bureau of Statistics (NBS) in collaboration with the World Bank. The paper draws on Wave 4 (2018/19) and Wave 5 (2023/24). Raw GHS-Panel microdata are publicly available from the World Bank Microdata Library (Wave 4: https://microdata.worldbank.org/index.php/catalog/3557 and Wave 5: https://microdata.worldbank.org/index.php/catalog/6410); free registration is required.

In addition, the analysis uses pre-processed LSMS panel data compiled by the Evans School Policy Analysis and Research Group (EPAR), available at:

https://github.com/EvansSchoolPolicyAnalysisAndResearch/LSMS-Data-Dissemination/tree/main/Nigeria%20GHS

Statement about Rights

I certify that the author(s) of the manuscript have legitimate access to and permission to use the data used in this manuscript.

Summary of Availability

☑ All data are publicly available.

☐ Some data cannot be made publicly available.

☐ No data can be made publicly available.

Both the World Bank GHS-Panel microdata and the EPAR processed files are freely and publicly accessible at the links above. Raw data files (`.dta`). Place World Bank GHS-Panel microdata Wave 4 files in `Source Data/Nigeria GHS Wave 5/Raw DTA files` and Wave 5 World Bank GHS-Panel microdata files in `Source Data/Nigeria GHS Wave 5/Raw DTA files` before running any code. Place the EPAR processed files into `Source Data/Nigeria GHS Wave #/final_data` for Wave 4 and 5 respectively (i.e. #=4 or #=5).

Getting Started

Outline and Folder Structure

The reproducibility package consists of five top-level folders:

- `Source Data/` — GHS survey files (download from the links above).
- `Stata Code/` — All Stata do-files for data cleaning and table production. Final analysis dataset is written to `Stata Code/Stata Data Landing/DML Cleaned Data.dta`.
- `R Code/` — Single script (`FIES Index Calculation.r`) for Rasch-model food insecurity index; called automatically by the Stata master.
- `Python Code/` — DML estimation, sensitivity analysis, and figure scripts. Outputs are written to `Python Code/Python Table Landing/` and `Tables and Figures/`.
- `Tables and Figures/` — Final paper outputs (`.tex`/`.csv` tables, `.png` figures).

```
$ROOT/
|-- 00_master.do                  # Stata master (Stage 1 entry point)
|-- 00_figures.py                 # Python figure master (Stage 2)
|-- requirements.txt             # Python package dependencies
|-- Source Data/Wave 4/           # GHS Wave 4 files
|-- Source Data/Wave 5/           # GHS Wave 5 files
|-- Stata Code/
|   |-- Stata Data Collection and Cleaning Scripts/
|   |    |-- Stata Control Covariate Collection and Cleaning Scripts/
|   |    |-- Stata Expenditure Calculations Scripts/
|   |    '--Stata Output Tables Scripts
|   '-- Stata Data Landing/DML Cleaned Data.dta  # Stage 1 output
|-- R Code/FIES Index Calculation.r
|-- Python Code/
|   |-- Python DML Scripts/       # fold_generator.py, learners.py,
|   |                             # helper_functions.py,
|   |                             # nuisance_function_residual_est.py,
|   |                             # DML Identification.py,
|   |                             # DML Benchmarking.py
|   |-- Pthon Figures Scripts/    # DML Figure A1, A2, A3, Benchmark, Summary
|   '-- Python Table Landing      # DML output location
'-- Tables and Figures/           # Final output landing page
```

How to Reproduce

PART 1: Stata — Data Construction. Open the `00_master.do` directly. Try to run the file without modification, the script attempts to construct paths automatically without input from the users. If an error is returned, comments inside should guide troubleshooting, but there are two primary issues that may arise:

1. If `00_master.do` returns an error that a Stata .dta file could not be found then either: 1) the data was not stored properly as outlined in Summary of Availability, or 2) the path macros are

not working correctly. In the latter case, `global root "‘c(pwd)’"` should automatically set the path for the root of the repoduction folder to the file's location. If this does not work then set it manually following the comments on lines 9-12.

2. If an error occurs because the Rscript does not run properly or because FIES dataset (`household_probs_long.csv`) cannot be found for Subsection 2.2 of `00_master.do`, then the do file is not correctly executing the Rscript. The script attempts to automatically find the Rscript executable path on your machine. However, this may not work for many users, and in this case you will need to set the path manually as described in the comments in `00_master.do` (lines 15-19).

This installs required Stata packages, calls the R FIES script, and produces `Stata Code/Stata Data Landing/DML Cleaned Data.dta`. All subsequent stages read from that file only. Estimated runtime: $\leq$ 1 min.

PART 2: Python — Install Dependencies. Install all required packages with a single command (see Section 6.2 for the full `requirements.txt`):

```
pip install -r requirements.txt
```

PART 3: Python — Estimation and Output. Run the following scripts in order. No path changes are needed; each script resolves paths relative to its own file location.

- **PART 3.1 — Descriptive figures (A1, A2, A3).** `00_figures.py` (run from `$ROOT`)
  Produces kernel density figures. Saved to `Tables and Figures/`.
- **PART 3.2 — Summary statistics.** `00_figures.py` (run from `$ROOT`)
  Produces farm-size quartile summary table; saved as `Tables and Figures/quartile_summary.csv`.
- **PART 3.3 — Main DML results.** `Python Code/Python DML Scripts/DML Identification.py`
  Estimates ATEs, HTEs (loan $\times$ farm size), and GATEs by farm-size quartile for each outcome. Output is a timestamped `.txt` file in `Python Table Landing/`. Set `N_REP=5` for a fast check; paper results use `N_REP=30`. Runtime: ≈20 min (`N_REP=5`); ≈2 hrs (2 Core Outcomes `N_REP=30`, 6 cores).
- **PART 3.4 — Ommited Variable Bias Benchmarking.** `Python Code/Python DML Scripts/DML Benchmarking.py`
  Computes omitted-variable bounds by re-estimating with each control dropped in turn. Output CSV files written to `Python Table Landing/`. Runtime: ≈36 hrs (2 Core Outcomes, 21 benchmarks, `N_REP=30`, 6 cores).
- **PART 3.5 — Sensitivity forest plot.** `Python Code/Python Figures Scripts/DML Figure Benchmark.py`
  Reads benchmark CSVs from PART 3.3/4 and saves the forest plot.

Cache behavior. After PART 3.3/4 runs once, nuisance predictions are cached in `Python Table Landing/nuisance_cache/`. Subsequent runs load the cache automatically (`re_estimate_nuisance=False` by default), which is much faster. Set `re_estimate_nuisance=True` only if you change the dataset, learners, or covariates.

Dataset List

Dataset	Description	Provided
`Source Data/Wave 4/Raw DTA Files/*.dta`	Raw GHS Wave 4 (2018/19) survey modules. (link above)	Yes
`Source Data/Wave 5/Raw DTA Files/*.dta`	Raw GHS Wave 5 (2023/24) survey modules. (link above)	Yes
`Source Data/Wave 4/final_data/*.dta`	EPAR LSMS processed data Wave 4 (linked above).	Yes
`Source Data/Wave 5/final_data/*.dta`	EPAR LSMS processed data Wave 5 (linked above).	Yes

List of Tables and Programs

Table / Figure	Program	Output file
Balance Table	`Stata Output Tables Scripts/ Balance Table.do`	`Tables and Figures/ balance_table.tex`
Main DML Results (ATE, HTE, GATE)	`Python DML Scripts/ DML Identification.py`	`Tables and Figures/ DML Identification_v*_*.txt`
Sensitivity Bounds	`Python DML Scripts/ DML Benchmarking.py`	`Tables and Figures/ benchmark_*.csv`
Summary Statistics	`Python Figures Scripts/ DML Summary Statistics.py`	`Tables and Figures/ quartile_summary_anova.csv`
Figure A1 (FIES KDE)	`Python Figures Scripts/ DML Figure A1.py`	`Tables and Figures/ Figure_A1.pdf`
Figure A2 (Farm-size KDE by treatment)	`Python Figures Scripts/ DML Figure A2.py`	`Tables and Figures/ Figure_A2.pdf`
Figure A3 (KDE by loan formality)	`Python Figures Scripts/ DML Figure A3.py`	`Tables and Figures/ Figure_A3.pdf`

Table / Figure	Program	Output file
Sensitivity Forest Plot	`Python Figures Scripts/ DML Figure Benchmark.py`	`Tables and Figures/ Figure_Benchmark.pdf`

Computational Requirements

Software Requirements

Stata

Stata/MP or Stata/SE version 17 or later. The following user-written packages are *automatically installed* by `00_master.do` on the first run:

Package	Source	Purpose
`univar`	SSC	Univariate statistics
`ietoolkit`	SSC	IE data management utilities
`winsor2`	SSC	Winsorizing continuous variables

R

R version 4.0 or later. Called via shell from `00_master.do` for the FIES index computation. R must be on the system `PATH` or its path must be set in the shell command in `00_master.do`. Required package: `RM.weights` (install via `install.packages("RM.weights")`).

Python

Python 3.9 or later (3.12 used for paper results). Table 3 lists all required packages with the exact versions used.

Table 3: Required Python packages

Package	Version	Purpose
`numpy`	1.26.4	Numerical arrays
`pandas`	2.2.2	Data manipulation; reads `.dta` via `read_stata`
`scikit-learn`	1.7.2	Ensemble learners and cross-validation
`doubleml`	0.11.1	DML Partial Linear Regression
`statsmodels`	0.14.6	Clustered OLS for GATE estimation
`matplotlib`	3.9.0	All figures
`scipy`	1.16.2	Kernel density estimation
`joblib`	1.5.2	Parallel nuisance estimation
`pyarrow`	22.0.0	Parquet I/O for nuisance cache

Installing Python Dependencies

Installation (run once from the repository root):

```
pip install -r requirements.txt
```

`requirements.txt:`

```
numpy==1.26.4
pandas==2.2.2
scikit-learn==1.7.2
doubleml==0.11.1
statsmodels==0.14.6
matplotlib==3.9.0
scipy==1.16.2
joblib==1.5.2
pyarrow==22.0.0
```

To install without pinned versions (use if the above causes conflicts):

```
pip install numpy pandas scikit-learn doubleml statsmodels matplotlib scipy joblib
    pyarrow
```

Key Configuration Parameters

Before running Stage 2, review these settings in `Python DML Scripts/DML Identification.py`:

Parameter	Paper value	Notes
`N_REP`	30	Set to $\leq$5 for a faster verification run
`k_fold_vector`	`[5]`	Cross-fitting folds
`N_WORKERS`	6	Set to available CPU cores
`re_estimate_nuisance`	`False`	`True` forces cache rebuild
`HTE`	`True`	Loan $\times$ farm-size interaction
`HTE_GATE`	`True`	Farm-size quartile (GATEs)

`N_REP` and `k_fold_vector` must be set to the same values in `DML Benchmarking.py`. The paper was run on Windows 11 with 6 physical cores and 16 GB RAM.

References

Lars Vilhuber, Connolly, M., Koren, M., Llull, J., & Morrow, P. (2022). "A template README for social science replication packages (v1.1)". *Zenodo*. https://doi.org/10.5281/zenodo.7293838

Evans School Policy Analysis and Research Group (EPAR). (2024). *LSMS Data Dissemination: Nigeria GHS*. https://github.com/EvansSchoolPolicyAnalysisAndResearch/LSMS-Data-Dissemination